Exercise: 01

Implement the Producer-Consumer problem using semaphores for synchronization. Instructions:

1) Write a program with multiple producer and consumer threads.

2) Use semaphores to ensure that producers wait when the buffer is full and consumers wait when the buffer is empty.

3) Ensure proper synchronization to avoid race conditions.

Exercise: 02

Solve the Dining Philosophers problem using mutexes. Instructions:

1) Write a program where five philosophers alternately think and eat.

2) Use mutexes to ensure that no two philosophers can eat simultaneously using the same fork.

3) Avoid deadlock and ensure fairness among philosophers.

Details:

Initialization:

• Define the number of philosophers (e.g., 5).

• Create an array of mutexes representing forks (one for each philosopher).

Philosopher's Actions:

• Each philosopher should alternately think and eat.

• When a philosopher wants to eat, they must pick up the left fork first and then the right fork (or vice versa) using mutexes.

Avoid Deadlock:

• To avoid deadlock, ensure that the philosophers pick up both forks in a consistent order (e.g., always pick up the

lower-numbered fork first).

• Alternatively, use a timeout mechanism to release the fork if the second fork cannot be acquired within a certain

period.

Ensure Fairness:

Ensure that all philosophers get a chance to eat by alternating between thinking and eating, and by using appropriate

synchronization mechanisms (e.g., a semaphore or mutex) to control access to the forks.

Philosopher Threads:

• Create a thread for each philosopher.

• Implement the actions for each philosopher in a loop where they think, try to pick up forks, eat, and then put down

the forks.